TecPro

TECNOLOGIAS

Desenvolvimento, Apostilas e Cursos Online



Linguagem



Listas



Árvores



Complexidade

Algoritmos e Estruturas de Dados

Em C++, com comparações em Python

Algoritmos e Estruturas de Dados

Em C++, com comparações em Python

TecPro Tecnologias

Desenvolvimento, Apostilas e Cursos Online

2026

Todos os direitos reservados. Nenhuma parte desta publicação pode ser reproduzida, armazenada ou transmitida de qualquer forma, ou por qualquer meio, sem autorização prévia da TecPro Tecnologias.

Sumário

Capítulo 1 – Fundamentos de C++ para Estruturas de Dados **4**
1.1 Por que estudar estruturas de dados em C++? . . . 4
1.2 Estrutura de um programa em C++ . . . 4
1.3 Tipos, variáveis e vetores . . . 5
1.4 Funções e passagem de parâmetros . . . 6
Por valor: a função recebe uma cópia . . . 6
Por referência: a função trabalha na variável original . . . 6
Por referência constante: sem cópia e sem alteração . . . 7
1.5 Ponteiros . . . 8
1.6 Memória dinâmica: new e delete . . . 9
1.7 Structs: agrupando dados . . . 10
Uma prévia: o nó de uma lista encadeada . . . 10
1.8 Classes: dados e comportamento juntos . . . 11
1.9 Templates: estruturas para qualquer tipo . . . 13
Erros Comuns . . . 13

Capítulo 2 – Complexidade de Algoritmos **15**
2.1 Por que medir a eficiência? . . . 15
2.2 Medir o tempo ou analisar o algoritmo? . . . 15
2.3 Contando operações . . . 16
2.4 Melhor caso, pior caso e caso médio . . . 16
2.5 A notação Big O . . . 17
A ideia: ficar com o termo que domina . . . 17
A definição formal . . . 17
2.6 As principais classes de complexidade . . . 18
2.7 Analisando código passo a passo . . . 19
Laço simples: O(n) . . . 19
Laços aninhados: $O(n^2)$ . . . 20
Laço que divide a entrada: O(log n) . . . 20
Laço que divide dentro de um laço linear: O(n log n) . . . 21
Recursão . . . 21
2.8 Complexidade de espaço . . . 22
Erros Comuns . . . 23

Capítulo 3 – Listas Encadeadas **24**
3.1 Quando o vetor não basta . . . 24
3.2 Tipo Abstrato de Dados . . . 24
3.3 O nó e a classe ListaEncadeada . . . 25
3.4 Inserindo elementos . . . 25
No início: O(1) . . . 25
No fim: O(1) graças ao ponteiro fim . . . 26
Em uma posição qualquer: O(n) . . . 27
3.5 Percorrendo e buscando . . . 27
3.6 Removendo elementos . . . 28
3.7 O programa completo . . . 29
3.8 Lista duplamente encadeada . . . 31
3.9 Lista circular . . . 34
3.10 Vetor ou lista encadeada? . . . 35
Erros Comuns . . . 37

Capítulo 4 – Pilhas e Filas **38**

4.1 Estruturas com regras de acesso 38
4.2 A pilha 38
4.3 Pilha com nós encadeados 39
4.4 Aplicações da pilha 40
Verificar se os parênteses estão balanceados 40
Calcular expressões em notação pós-fixa 41
A pilha de execução 43
4.5 A fila 44
4.6 Fila com nós encadeados 44
4.7 Fila circular em vetor 46
4.8 Aplicações da fila 47
4.9 Pilhas e filas prontas 48
Erros Comuns 49

Capítulo 1 – Fundamentos de C++ para Estruturas de Dados

Em Algoritmos I você aprendeu a base da programação: variáveis, tipos e operadores, condicionais e laços, funções, strings, vetores e matrizes. Também conheceu, de forma introdutória, listas, pilhas, filas, busca, ordenação e recursão. Agora vamos aprofundar: construir essas estruturas por dentro, de forma eficiente, e analisar o custo de cada operação.

Para isso, você precisa de algumas ferramentas do C++ que as estruturas de dados usam o tempo todo. Este capítulo faz duas coisas:

- **retoma rapidamente** o que você já conhece, agora escrito em C++: estrutura de um programa, tipos, vetores e funções (seções 1.2 a 1.4);
- **apresenta o que é novo** e indispensável daqui em diante: ponteiros, memória dinâmica, structs, classes e templates (seções 1.5 a 1.9).

Se você já se sente seguro com a primeira parte, pode passar por ela rapidamente. A segunda parte merece atenção total: é a base de todos os capítulos seguintes.

1.1 Por que estudar estruturas de dados em C++?

Em linguagens como Python, Java ou JavaScript, a memória é administrada automaticamente. Você cria uma lista, adiciona elementos e nunca precisa pensar onde eles estão guardados. Isso é ótimo para produtividade, mas esconde justamente o que queremos entender nesta disciplina: **como os dados são organizados na memória**.

Em C++, você enxerga e controla essa organização. Uma lista encadeada deixa de ser uma caixa-preta e passa a ser o que ela realmente é: um conjunto de blocos de memória, cada um apontando para o próximo. Quando você entende isso, passa a entender também por que uma operação é rápida ou lenta, e esse é o centro da análise de algoritmos.

Ao longo do livro, sempre que ajudar a entender um conceito, vamos mostrar como a mesma ideia aparece em Python. Assim você vê as duas faces da moeda: o que acontece *por dentro* (C++) e como isso costuma ficar *escondido* em uma linguagem de mais alto nível (Python).

1.2 Estrutura de um programa em C++

Todo programa em C++ começa a ser executado pela função `main`. Veja um programa completo:

```
#include <iostream>   // biblioteca de entrada e saída (cin, cout)
using namespace std;  // permite escrever cout em vez de std::cout

int main() {
    int idade;
    cout << "Digite sua idade: ";
    cin >> idade;                       // lê um valor do teclado
```

```
    cout << "Daqui a 10 anos voce tera " << idade + 10 << " anos" <<
    endl;
    return 0;                              // 0 indica que terminou sem
    erro
}
```

Alguns pontos importantes:

- `#include` traz uma biblioteca para o programa. A `iostream` oferece `cin` (entrada) e `cout` (saída).
- Todo comando termina com `;` e todo bloco fica entre `{` e `}`.
- C++ diferencia letras maiúsculas de minúsculas: `idade` e `Idade` são variáveis diferentes.
- O programa precisa ser **compilado** antes de rodar. Com o compilador g++, por exemplo: `g++ programa.cpp -o programa`.

Comparando com Python

Em Python, o mesmo programa não precisa de `main`, de tipos nem de compilação, porque é interpretado linha a linha:

```
idade = int(input("Digite sua idade: "))
print(f"Daqui a 10 anos voce tera {idade + 10} anos")
```

Em C++, o compilador verifica os tipos antes de o programa rodar. Muitos erros que em Python só aparecem durante a execução, em C++ aparecem já na compilação.

1.3 Tipos, variáveis e vetores

Uma variável é um espaço na memória com um nome e um tipo. O tipo define quanto espaço ela ocupa e que valores pode guardar.

Tipo	Guarda	Exemplo
`int`	números inteiros	`int qtd = 10;`
`double`	números com casas decimais	`double nota = 8.5;`
`char`	um caractere	`char letra = 'A';`
`bool`	verdadeiro ou falso	`bool ok = true;`
`string`	texto (precisa de `<string>`)	`string nome = "Ana";`

Um **vetor** (array) guarda vários valores do mesmo tipo, um ao lado do outro na memória. Os índices começam em 0:

```
int notas[5] = {7, 9, 6, 10, 8};   // 5 inteiros seguidos na memória

for (int i = 0; i < 5; i++) {
    cout << "Nota " << i << ": " << notas[i] << endl;
}
```

O vetor tem uma limitação que vai motivar vários capítulos deste livro: o **tamanho é fixo**. Se você declarou espaço para 5 notas e chegou uma sexta, não há onde colocá-la. Inserir um elemento no meio também é trabalhoso, porque todos os seguintes precisam ser deslocados uma posição. As estruturas que vamos construir, como as listas encadeadas, existem justamente para contornar essas limitações.

1.4 Funções e passagem de parâmetros

Uma função é um trecho de código com nome, que recebe dados (parâmetros), faz um trabalho e pode devolver um resultado. Funções evitam repetir código e deixam o programa mais fácil de ler e de manter.

```
// recebe os três coeficientes e diz se a equação tem raízes reais
bool temRaizReal(double a, double b, double c) {
    double delta = b * b - 4 * a * c;
    return delta >= 0;
}

// void: a função não devolve valor, só executa uma ação
void imprimirVetor(int v[], int tamanho) {
    for (int i = 0; i < tamanho; i++) {
        cout << v[i] << " ";
    }
    cout << endl;
}
```

O ponto mais importante para este livro é **como os parâmetros chegam à função**. Em C++ há três formas.

Por valor: a função recebe uma cópia

```
void dobrar(int x) {
    x = x * 2;          // altera só a cópia
}

int main() {
    int n = 5;
    dobrar(n);
    cout << n << endl;  // imprime 5: o n original não mudou
}
```

Por referência: a função trabalha na variável original

Basta acrescentar `&` depois do tipo do parâmetro. A função passa a usar a própria variável de quem chamou, sem cópia:

```
void dobrar(int &x) {
    x = x * 2;          // altera a variável original
}
```

```
void trocar(int &a, int &b) {
    int auxiliar = a;
    a = b;
    b = auxiliar;
}

int main() {
    int n = 5;
    dobrar(n);
    cout << n << endl;  // imprime 10

    int p = 1, q = 2;
    trocar(p, q);
    cout << p << " " << q << endl;  // imprime 2 1
}
```

Por referência constante: sem cópia e sem alteração

Quando o dado é grande (um texto longo, um vetor de milhares de itens) e a função só precisa ler, use `const &`. Você evita a cópia, que custa tempo e memória, e o compilador garante que a função não altera o original:

```
#include <string>

int contarVogais(const string &texto) {
    int total = 0;
    for (char c : texto) {
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
        {
            total++;
        }
    }
    return total;
}
```

Comparando com Python

Python não deixa você escolher. Números e textos se comportam como se fossem passados por valor, porque não podem ser alterados (são *imutáveis*). Já listas e dicionários são passados de forma que a função pode modificá-los:

```
def dobrar(x):
    x = x * 2          # cria um novo número só dentro da função

def adicionar(lista):
    lista.append(99)   # modifica a lista original

n = 5
dobrar(n)
```

```
print(n)       # 5

numeros = [1, 2]
adicionar(numeros)
print(numeros) # [1, 2, 99]
```

Em C++ essa decisão é sua e fica escrita na assinatura da função, o que torna o comportamento explícito.

1.5 Ponteiros

Toda variável ocupa uma posição na memória, e cada posição tem um **endereço**, como o número de uma casa em uma rua. Um **ponteiro** é uma variável que guarda um endereço: em vez de guardar um valor, ele aponta para onde o valor está.

Dois operadores fazem todo o trabalho:

- `&variavel` devolve o **endereço** da variável;
- `*ponteiro` acessa o **valor** que está no endereço guardado (isso se chama *desreferenciar*).

```
int idade = 20;
int *p = &idade;      // p guarda o endereço de idade

cout << idade << endl;  // 20
cout << p << endl;      // um endereço, por exemplo 0x61fe14
cout << *p << endl;     // 20: o valor que está no endereço

*p = 21;                // altera idade através do ponteiro
cout << idade << endl;  // 21
```

Uma forma de visualizar o que aconteceu na memória:

Variável	Endereço	Conteúdo
`idade`	`0x61fe14`	`21`
`p`	`0x61fe18`	`0x61fe14` (aponta para `idade`)

Um ponteiro que não aponta para nada deve valer `nullptr`. Você vai usar muito esse valor: é ele que marca o fim de uma lista encadeada ou um galho vazio de uma árvore. Antes de desreferenciar um ponteiro, verifique se ele não é nulo:

```
int *q = nullptr;   // não aponta para nada

if (q != nullptr) {
    cout << *q << endl;  // só acessa se for seguro
}
```

Referência ou ponteiro?

Os dois permitem trabalhar com a variável original, mas têm diferenças. Uma **referência** (`int &r = x;`) é um apelido: precisa nascer ligada a uma variável e nunca muda de alvo. Um **ponteiro** pode começar vazio (`nullptr`) e pode passar a apontar para outro lugar. Por isso as estruturas de dados usam ponteiros: um nó de uma lista precisa poder apontar para *nenhum* nó, ou mudar de vizinho quando você insere ou remove elementos.

1.6 Memória dinâmica: new e delete

As variáveis declaradas dentro de uma função ficam em uma área chamada **pilha de execução** (*stack*). Elas são criadas quando a função começa e destruídas automaticamente quando ela termina. Isso é prático, mas tem um limite: você não consegue criar, durante a execução, quantos elementos quiser e mantê-los vivos depois que a função acabar.

Para isso existe a memória dinâmica, ou **heap**. Com `new`, você pede ao sistema um espaço que continua existindo até você mesmo liberá-lo com `delete`:

```
int *p = new int;     // reserva espaço para um int no heap
*p = 42;
cout << *p << endl;   // 42
delete p;             // devolve o espaço
p = nullptr;          // boa prática: não deixar o endereço antigo

// vetor com tamanho decidido durante a execução
int n;
cout << "Quantos valores? ";
cin >> n;
int *valores = new int[n];   // n inteiros no heap
for (int i = 0; i < n; i++) {
    valores[i] = i * i;
}
delete[] valores;            // vetor: libera com delete[]
```

Toda estrutura de dados dinâmica deste livro funciona assim: cada novo elemento é criado com `new` e, quando sai da estrutura, é liberado com `delete`.

Regra de ouro da memória dinâmica

Para cada `new` deve existir exatamente um `delete` (e para cada `new[]`, um `delete[]`). Se você esquecer o `delete`, o espaço fica preso até o programa terminar: é o chamado **vazamento de memória** (*memory leak*). Se fizer `delete` duas vezes, ou usar o ponteiro depois do `delete`, o comportamento é imprevisível e o programa pode travar.

Comparando com Python

Em Python não existe `delete` para liberar memória. O próprio interpretador percebe quando um objeto não é mais usado por ninguém e o remove: é a **coleta de lixo** (*garbage collection*). Isso evita vazamentos e ponteiros inválidos, mas tem um custo:

você perde o controle de quando a memória é liberada e de quanto espaço cada estrutura realmente ocupa.

1.7 Structs: agrupando dados

Uma `struct` junta vários dados relacionados em um único tipo novo. Em vez de ter variáveis soltas para nome, idade e matrícula de um aluno, você cria o tipo `Aluno`:

```
struct Aluno {
    string nome;
    int matricula;
    double media;
};

int main() {
    Aluno a;
    a.nome = "Ana";          // o ponto acessa cada campo
    a.matricula = 2026001;
    a.media = 8.7;

    Aluno *p = &a;
    cout << p->nome << endl; // com ponteiro, usa-se a seta ->
}
```

Repare no operador `->`: quando você tem um **ponteiro** para uma struct, `p->nome` é o mesmo que `(*p).nome`. Ele vai aparecer em praticamente todo o código dos próximos capítulos.

Uma prévia: o nó de uma lista encadeada

Uma struct pode ter um campo que é ponteiro para o próprio tipo. Essa ideia simples é a base das listas encadeadas e das árvores:

```
struct No {
    int valor;
    No *proximo;   // aponta para o próximo nó (ou nullptr)
};

int main() {
    // cria três nós no heap
    No *a = new No{10, nullptr};
    No *b = new No{20, nullptr};
    No *c = new No{30, nullptr};

    // liga os nós: a -> b -> c
    a->proximo = b;
    b->proximo = c;

    // percorre a cadeia a partir do primeiro nó
    for (No *atual = a; atual != nullptr; atual = atual->proximo) {
```

```
        cout << atual->valor << " ";   // 10 20 30
    }
    cout << endl;

    delete a;
    delete b;
    delete c;
}
```

Os três nós podem estar em qualquer lugar da memória, não precisam estar lado a lado como num vetor. O que os mantém em ordem são os ponteiros. Por isso, para inserir um valor no meio, basta mudar dois ponteiros, sem deslocar nenhum elemento. No Capítulo 3, essa ideia vira uma lista completa.

1.8 Classes: dados e comportamento juntos

Uma classe vai além da struct: ela reúne os dados e as funções que trabalham com eles, e controla quem pode acessar o quê. Esse controle se chama **encapsulamento**. Os dados ficam em `private`, protegidos de alterações indevidas, e o acesso acontece por funções públicas, os **métodos**.

Como exemplo, veja uma classe que resolve o problema do tamanho fixo dos vetores: um vetor que cresce sozinho quando enche.

```
class VetorDinamico {
private:
    int *dados;        // vetor no heap
    int tamanho;       // quantos elementos estão em uso
    int capacidade;    // quantos cabem antes de crescer

    void crescer() {
        capacidade = capacidade * 2;
        int *novo = new int[capacidade];
        for (int i = 0; i < tamanho; i++) {
            novo[i] = dados[i];      // copia os elementos antigos
        }
        delete[] dados;              // libera o vetor antigo
        dados = novo;
    }

public:
    // construtor: executado quando o objeto é criado
    VetorDinamico() {
        capacidade = 2;
        tamanho = 0;
        dados = new int[capacidade];
    }

    // destrutor: executado quando o objeto deixa de existir
    ~VetorDinamico() {
        delete[] dados;
```

```
    }

    void adicionar(int valor) {
        if (tamanho == capacidade) {
            crescer();
        }
        dados[tamanho] = valor;
        tamanho++;
    }

    int obter(int indice) const {
        return dados[indice];
    }

    int quantidade() const {
        return tamanho;
    }
};

int main() {
    VetorDinamico v;
    for (int i = 1; i <= 10; i++) {
        v.adicionar(i * 10);   // cresce sozinho: 2, 4, 8, 16 posições
    }
    for (int i = 0; i < v.quantidade(); i++) {
        cout << v.obter(i) << " ";
    }
    cout << endl;
}   // aqui o destrutor libera a memória automaticamente
```

Repare em três ideias que vão se repetir ao longo do livro:

- **Construtor** (`VetorDinamico()`): prepara o objeto quando ele nasce. Tem o mesmo nome da classe e não tem tipo de retorno.
- **Destrutor** (`~VetorDinamico()`): libera os recursos quando o objeto deixa de existir. Assim, quem usa a classe não precisa se lembrar de chamar `delete`.
- **Métodos `const`**: `obter` e `quantidade` apenas consultam o objeto, e o `const` garante que não alteram nada.

Todas as estruturas dos próximos capítulos (lista, pilha, fila, árvore) serão classes com esse formato: dados privados, métodos públicos para as operações e um destrutor que libera os nós.

Comparando com Python

A lista do Python (`list`) funciona exatamente como o `VetorDinamico`: por dentro, ela reserva espaço sobrando e, quando enche, cria um espaço maior e copia os elementos. Você só não vê esse trabalho. A mesma classe em Python ficaria assim:

```
class Contador:
    def __init__(self):        # construtor
```

```
        self._total = 0        # o _ indica "uso interno"

    def incrementar(self):
        self._total += 1

    def valor(self):
        return self._total
```

Em Python, o `private` é só uma convenção (o sublinhado), e não existe destrutor para liberar memória, porque a coleta de lixo cuida disso.

1.9 Templates: estruturas para qualquer tipo

O `VetorDinamico` só guarda `int`. Se você precisar de um vetor de `double` ou de `string`, teria de copiar a classe inteira trocando o tipo. Os **templates** resolvem isso: você escreve o código uma vez, com um tipo genérico `T`, e o compilador gera a versão de cada tipo usado.

```
// função genérica: funciona com int, double, string...
template <typename T>
T maior(T a, T b) {
    return (a > b) ? a : b;
}

// struct genérica: o nó pode guardar qualquer tipo
template <typename T>
struct NoGenerico {
    T valor;
    NoGenerico<T> *proximo;
};

int main() {
    cout << maior(3, 7) << endl;                          // 7
    cout << maior(2.5, 1.5) << endl;                      // 2.5
    cout << maior(string("ana"), string("bia")) << endl;  // bia

    NoGenerico<string> *n = new NoGenerico<string>{"oi", nullptr};
    cout << n->valor << endl;
    delete n;
}
```

A biblioteca padrão do C++ (STL) já traz estruturas prontas feitas com templates, como `vector`, `list`, `stack`, `queue` e `map`. Neste livro vamos construir as nossas versões, porque o objetivo é entender como elas funcionam por dentro. Depois de entender, use as da STL nos seus projetos: são testadas e otimizadas.

Erros Comuns

- **Usar um ponteiro não inicializado.** Declarar `int *p;` e depois fazer `*p = 5;` escreve em um endereço aleatório. Inicialize sempre: com um endereço válido

ou com `nullptr`.

- **Desreferenciar `nullptr`.** Acessar `p->valor` quando `p` é nulo derruba o programa. Em listas e árvores, confira se o ponteiro não é nulo antes de usá-lo.
- **Esquecer o `delete`.** Cada `new` sem o `delete` correspondente é memória perdida. Em classes, libere no destrutor.
- **Usar `delete` em vez de `delete[]`.** O que foi criado com `new[]` deve ser liberado com `delete[]`.
- **Usar o ponteiro depois do `delete`.** O espaço já foi devolvido, e o endereço antigo não é mais seu. Depois de liberar, faça `p = nullptr;`.
- **Esperar que uma função altere um parâmetro passado por valor.** Se a função precisa modificar a variável de quem chamou, use referência (`&`) ou ponteiro.
- **Acessar um vetor fora dos limites.** Em um vetor de 5 posições, os índices vão de 0 a 4. C++ não avisa quando você acessa `v[5]`: o programa simplesmente lê ou estraga outra área da memória.

Capítulo 2 – Complexidade de Algoritmos

Dois programas podem resolver o mesmo problema e dar a mesma resposta, mas um terminar em um piscar de olhos e o outro levar horas. A diferença quase nunca está no computador: está no algoritmo. Neste capítulo você vai aprender a medir a eficiência de um algoritmo sem precisar executá-lo, usando a notação Big O. Essa ferramenta vai acompanhar você em todos os capítulos seguintes: para cada estrutura de dados, vamos perguntar quanto custa inserir, buscar e remover.

2.1 Por que medir a eficiência?

Imagine um sistema com o cadastro de 1 milhão de clientes. Para encontrar um cliente pelo CPF, você pode olhar um por um, do primeiro ao último. No pior caso, são 1 milhão de comparações. Se o cadastro estiver ordenado, dá para fazer como numa lista telefônica: abrir no meio, ver se o CPF está antes ou depois e descartar metade a cada passo. Assim, bastam cerca de 20 comparações.

Os dois métodos funcionam, mas um é 50 mil vezes mais rápido que o outro. E a diferença aumenta conforme o cadastro cresce. É por isso que analisamos algoritmos: para escolher, antes de programar, a solução que continua viável quando os dados crescem.

A eficiência pode ser medida em dois recursos:

- **Tempo**: quantas operações o algoritmo faz. É a análise mais comum e o foco deste capítulo.
- **Espaço**: quanta memória extra o algoritmo precisa. Veremos na seção 2.8.

2.2 Medir o tempo ou analisar o algoritmo?

A forma mais direta de saber se um programa é rápido é executá-lo e cronometrar. Em C++, a biblioteca `chrono` faz isso:

```
#include <chrono>

long long somar(int n) {
    long long soma = 0;
    for (int i = 1; i <= n; i++) {
        soma += i;
    }
    return soma;
}

int main() {
    auto inicio = chrono::steady_clock::now();
    long long resultado = somar(100000000);
    auto fim = chrono::steady_clock::now();

    auto ms = chrono::duration_cast<chrono::milliseconds>(fim -
    inicio).count();
    cout << "Resultado: " << resultado << endl;
    cout << "Tempo: " << ms << " ms" << endl;
```

```
}
```

Essa **análise experimental** é útil, mas tem limites. O tempo medido depende do processador, do sistema operacional, do compilador, de outros programas abertos e até dos dados usados no teste. Rodando o mesmo código em outra máquina, o número muda. E, para medir, você precisa ter o algoritmo pronto.

A **análise de complexidade** olha para o algoritmo em si. Em vez de medir segundos, ela conta **quantas operações** o algoritmo faz em função do tamanho da entrada. O resultado vale para qualquer máquina e pode ser obtido antes de escrever o código. As duas análises se completam: a de complexidade ajuda a escolher o algoritmo, e a experimental ajusta os detalhes quando o tempo é crítico.

2.3 Contando operações

O primeiro passo da análise é definir o **tamanho da entrada**, que chamamos de n. Ele depende do problema:

Problema	Tamanho da entrada (n)
Ordenar ou buscar em um vetor	quantidade de elementos
Operações em uma lista ou árvore	quantidade de nós
Algoritmos em grafos	número de vértices e de arestas
Processar um texto	quantidade de caracteres

Depois, contamos as operações que importam. Veja a função que soma os elementos de um vetor:

```
int somarVetor(int v[], int n) {
    int soma = 0;               // 1 vez
    for (int i = 0; i < n; i++) {
        soma += v[i];           // n vezes
    }
    return soma;                // 1 vez
}
```

A atribuição inicial e o `return` acontecem uma vez; a soma dentro do laço acontece n vezes. Contando também as comparações e incrementos do `for`, chegamos a algo como $t(n) = 3n + 3$ operações. O número exato depende de como se conta, e é aí que entra a ideia mais importante do capítulo: **o que interessa não é o valor exato, e sim como $t(n)$ cresce quando n cresce**. Se o vetor dobrar de tamanho, o trabalho dobra. Esse comportamento é o que a notação Big O captura.

2.4 Melhor caso, pior caso e caso médio

Para muitos algoritmos, o número de operações depende não só do tamanho, mas também do **conteúdo** da entrada. A busca sequencial é o exemplo clássico:

```
// devolve a posição do valor procurado, ou -1 se não encontrar
int buscaSequencial(int v[], int n, int procurado) {
    for (int i = 0; i < n; i++) {
        if (v[i] == procurado) {
            return i;
        }
    }
    return -1;
}
```

- **Melhor caso**: o valor está na primeira posição. Uma comparação basta, não importa o tamanho do vetor.
- **Pior caso**: o valor está na última posição ou não existe. São n comparações.
- **Caso médio**: supondo que o valor esteja no vetor e que qualquer posição seja igualmente provável, em média percorremos metade do vetor, cerca de $n/2$ comparações.

O **pior caso** é o mais usado, por um motivo prático: ele é uma **garantia**. Se o pior caso de um algoritmo é aceitável, ele nunca vai decepcionar. O melhor caso costuma ser otimista demais para servir de base a uma decisão, e o caso médio exige saber como os dados se distribuem, o que nem sempre é possível.

2.5 A notação Big O

A notação Big O descreve como o tempo de um algoritmo cresce, deixando de lado os detalhes que não mudam esse crescimento. Dizemos que a busca sequencial é $O(n)$ (lê-se "ó de ene"): no pior caso, o tempo cresce de forma proporcional ao tamanho da entrada.

A ideia: ficar com o termo que domina

Suponha que um algoritmo faça $t(n) = 3n^2 + 5n + 20$ operações. Veja quanto cada parte contribui:

n	$3n^2$	$5n$	20	**parte de $3n^2$**
10	300	50	20	81%
100	30.000	500	20	98,3%
1.000	3.000.000	5.000	20	99,8%

Quando n cresce, o termo n^2 domina completamente. As constantes (o 3, o 5, o 20) dependem de detalhes de implementação e da máquina. Por isso, a notação Big O fica só com o termo dominante, sem a constante: esse algoritmo é $O(n^2)$.

A definição formal

Uma função $t(n)$ é $O(f(n))$ se existem constantes positivas c e k tais que

$$t(n) \leq c \cdot f(n) \quad \text{para todo } n \geq k.$$

Em palavras: a partir de um certo tamanho de entrada (k), o tempo do algoritmo nunca passa de um múltiplo (c) de $f(n)$. No exemplo acima, com $c = 4$ e $k = 8$, temos $3n^2 + 5n + 20 \leq 4n^2$ para todo $n \geq 8$; logo $t(n)$ é $O(n^2)$.

Regras práticas do Big O

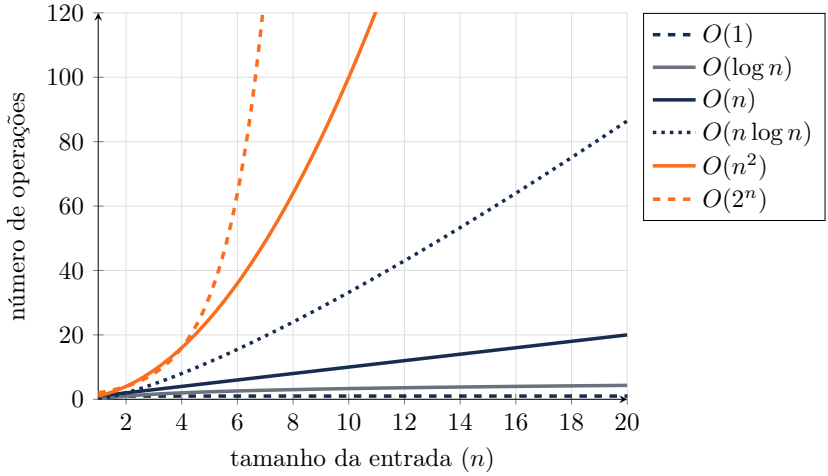
- **Descarte as constantes**: $O(3n)$ é $O(n)$; $O(n^2/2)$ é $O(n^2)$.
- **Fique com o termo de maior crescimento**: $O(n^2+n)$ é $O(n^2)$; $O(n+\log n)$ é $O(n)$.
- **Trechos em sequência se somam**: um laço $O(n)$ seguido de outro $O(n^2)$ resulta em $O(n^2)$.
- **Trechos aninhados se multiplicam**: um laço de n voltas dentro de outro de n voltas resulta em $O(n \cdot n) = O(n^2)$.

2.6 As principais classes de complexidade

Quase todos os algoritmos que você vai encontrar se encaixam em poucas classes:

Classe	Nome	Exemplo
$O(1)$	constante	acessar `v[i]`; inserir no início de uma lista encadeada
$O(\log n)$	logarítmica	busca binária em vetor ordenado
$O(n)$	linear	busca sequencial; somar um vetor
$O(n \log n)$	linearítmica	merge sort; heapsort
$O(n^2)$	quadrática	bubble sort; comparar todos os pares
$O(n^3)$	cúbica	multiplicação simples de matrizes
$O(2^n)$	exponencial	testar todos os subconjuntos de um conjunto

O gráfico mostra como essas funções crescem. Repare como $O(\log n)$ quase não sai do chão, enquanto $O(2^n)$ dispara logo no início:



Para sentir a diferença na prática, imagine um computador que faz **1 milhão de operações por segundo**. Quanto tempo cada classe levaria?

Classe	$n = 1.000$	$n = 1.000.000$
$O(\log n)$	10 microssegundos	20 microssegundos
$O(n)$	1 milissegundo	1 segundo
$O(n \log n)$	10 milissegundos	20 segundos
$O(n^2)$	1 segundo	cerca de 11,6 dias
$O(2^n)$	mais que a idade do universo	impossível

A tabela explica por que a escolha do algoritmo importa mais que a velocidade do computador. Um processador mil vezes mais rápido transforma os 11,6 dias do $O(n^2)$ em cerca de 17 minutos, mas um algoritmo $O(n \log n)$, na máquina antiga, resolve o mesmo problema em 20 segundos.

2.7 Analisando código passo a passo

Com as regras da seção 2.5, dá para descobrir a complexidade lendo o código. Veja os padrões mais comuns.

Laço simples: O(n)

```
// encontra o maior valor do vetor: percorre cada elemento uma vez
int maiorValor(int v[], int n) {
    int maior = v[0];
    for (int i = 1; i < n; i++) {
        if (v[i] > maior) {
            maior = v[i];
        }
```

```
    }
    return maior;
}
```

Laços aninhados: O(n²)

```
// verifica se o vetor tem algum valor repetido, comparando todos os
pares
bool temRepetido(int v[], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (v[i] == v[j]) {
                return true;
            }
        }
    }
    return false;
}
```

O laço interno começa em `i + 1`, então ele dá $n - 1$ voltas, depois $n - 2$, e assim por diante. A soma é $n(n-1)/2$ comparações no pior caso. Descartando a constante $1/2$ e o termo menor, a função é $O(n^2)$.

Laço que divide a entrada: O(log n)

```
// quantas vezes n pode ser dividido por 2 até chegar a 1?
int contarDivisoes(int n) {
    int passos = 0;
    while (n > 1) {
        n = n / 2;
        passos++;
    }
    return passos;
}

int main() {
    cout << contarDivisoes(1000) << endl;     // 9
    cout << contarDivisoes(1000000) << endl;  // 19
}
```

Sempre que um algoritmo **descarta metade** do problema a cada passo, ele é logarítmico. Multiplicar a entrada por mil acrescenta só uns 10 passos. É o segredo da busca binária (Capítulo 7) e das árvores balanceadas (Capítulo 5).

Laço que divide dentro de um laço linear: O(n log n)

```
int exemploNLogN(int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {           // n vezes
        for (int j = n; j > 1; j = j / 2) { // log n vezes
            total++;
        }
    }
    return total;
}
```

Esse é o padrão dos bons algoritmos de ordenação, como o merge sort (Capítulo 8): cada um dos $\log n$ níveis de divisão faz um trabalho linear.

Recursão

Em Algoritmos I você viu que uma função recursiva chama a si mesma. Para analisá-la, conte quantas chamadas acontecem e quanto trabalho cada uma faz.

```
// fatorial: n chamadas, cada uma com trabalho constante -> O(n)
long long fatorial(int n) {
    if (n <= 1) return 1;
    return n * fatorial(n - 1);
}

// Fibonacci recursivo: cada chamada gera outras duas -> O(2^n)
long long fibRecursivo(int n) {
    if (n <= 1) return n;
    return fibRecursivo(n - 1) + fibRecursivo(n - 2);
}

// Fibonacci com laço: guarda só os dois últimos valores -> O(n)
long long fibIterativo(int n) {
    long long anterior = 0, atual = 1;
    for (int i = 0; i < n; i++) {
        long long proximo = anterior + atual;
        anterior = atual;
        atual = proximo;
    }
    return anterior;
}

int main() {
    cout << fatorial(10) << endl;      // 3628800
    cout << fibRecursivo(30) << endl;  // 832040, após mais de 1
    milhão de chamadas
    cout << fibIterativo(30) << endl;  // 832040, com apenas 30 voltas
}
```

O Fibonacci recursivo é um ótimo alerta: o código é curto e elegante, mas recalcula os mesmos valores muitas vezes. Para `fibRecursivo(50)`, seriam bilhões de chamadas.

A versão com laço resolve em 50 passos. No Capítulo 9 você vai ver a programação dinâmica, uma técnica que conserta exatamente esse tipo de desperdício.

Comparando com Python

Em Python, operações que parecem instantâneas podem esconder um laço inteiro. Saber a complexidade delas evita surpresas:

```
nomes = ["ana", "bia", "caio", "davi"]

print("davi" in nomes)   # O(n): percorre a lista até achar
nomes.append("eva")      # O(1) em média: coloca no fim
nomes.insert(0, "zoe")   # O(n): desloca todos os elementos
nomes.pop()              # O(1): remove do fim
nomes.pop(0)             # O(n): desloca todos os elementos

conjunto = set(nomes)
print("davi" in conjunto)  # O(1) em média: usa tabela hash
```

Uma linha como `if x in lista` dentro de um laço transforma um algoritmo que parecia $O(n)$ em $O(n^2)$. Em C++, como você escreve os laços, esse custo fica visível no próprio código.

2.8 Complexidade de espaço

A mesma notação mede a **memória extra** que o algoritmo usa, além da própria entrada.

- `somarVetor` e `fibIterativo` usam só algumas variáveis, não importa o tamanho de n: espaço $O(1)$.
- Um algoritmo que cria uma cópia do vetor para trabalhar nela usa espaço $O(n)$.
- O `fatorial` recursivo também usa espaço $O(n)$, mesmo sem criar vetores: cada chamada que ainda não terminou ocupa um espaço na pilha de execução, e há n chamadas abertas ao mesmo tempo.

Muitas vezes é possível trocar espaço por tempo: guardar resultados já calculados gasta memória, mas evita refazer contas. Essa troca é a base da programação dinâmica.

Para saber mais: Ômega e Theta

Além do Big O, existem outras duas notações. A notação Ω (ômega) indica um **limite inferior**: o algoritmo faz *pelo menos* aquela quantidade de trabalho. A notação Θ (theta) indica um **limite exato**: o crescimento é *exatamente* daquela ordem, nem mais nem menos. Na prática, quando alguém diz "a busca sequencial é $O(n)$", costuma querer dizer que o pior caso cresce exatamente como n. Neste livro, seguindo o uso comum, usaremos o Big O.

Erros Comuns

- **Confundir Big O com tempo em segundos.** $O(n)$ não diz quanto tempo o programa leva, e sim como esse tempo cresce. Um algoritmo $O(n)$ pode ser mais lento que um $O(n^2)$ para entradas pequenas.
- **Manter constantes e termos menores.** Escrever $O(2n + 10)$ está tecnicamente certo, mas o correto é simplificar para $O(n)$.
- **Somar quando deveria multiplicar.** Dois laços de n voltas *um depois do outro* dão $O(n)$; *um dentro do outro* dão $O(n^2)$.
- **Esquecer o custo das operações de biblioteca.** Chamar uma função que percorre a estrutura inteira (como `in` em uma lista do Python) dentro de um laço multiplica a complexidade.
- **Analisar só o melhor caso.** Dizer que a busca sequencial é $O(1)$ porque às vezes o valor está na primeira posição esconde o comportamento real. Use o pior caso como referência.
- **Ignorar o espaço da recursão.** Uma função recursiva com n chamadas abertas ao mesmo tempo usa memória $O(n)$, e com n muito grande pode estourar a pilha de execução.

Capítulo 3 – Listas Encadeadas

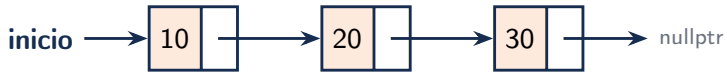
Em Algoritmos I você trabalhou com vetores: uma sequência de posições lado a lado na memória, com tamanho definido na criação. Neste capítulo você vai construir uma estrutura que cresce e encolhe conforme a necessidade, sem desperdiçar espaço e sem deslocar elementos: a **lista encadeada**. Ela é a primeira estrutura dinâmica do livro e a base de várias outras, como pilhas, filas e grafos.

3.1 Quando o vetor não basta

Imagine um sistema que registra os pedidos de uma loja ao longo do dia. Não dá para saber quantos pedidos vão chegar. Se você criar um vetor de 1.000 posições e chegarem 30 pedidos, desperdiçou memória; se chegarem 1.500, faltou espaço. O vetor dinâmico do Capítulo 1 resolve parte do problema, mas cada vez que enche precisa copiar tudo para um espaço maior.

Há um segundo problema: inserir ou remover **no meio**. Para colocar um pedido urgente na frente da fila de um vetor, todos os outros precisam andar uma posição. Com 1 milhão de pedidos, isso é 1 milhão de movimentações.

A lista encadeada resolve as duas questões com uma ideia simples: cada elemento fica em um **nó** criado separadamente na memória, e cada nó guarda, além do valor, um **ponteiro para o próximo**. Os nós podem estar em qualquer lugar da memória; o que os mantém em ordem são os ponteiros.



cada nó guarda um **valor** e um ponteiro para o **próximo**

A lista só precisa guardar onde está o primeiro nó (`inicio`). A partir dele, seguindo os ponteiros, chega-se a todos os outros. O último nó aponta para `nullptr`, que marca o fim.

3.2 Tipo Abstrato de Dados

Antes de programar, vale separar duas perguntas: *o que* a lista faz e *como* ela faz. A resposta à primeira pergunta é a **interface**: o conjunto de operações que quem usa a lista pode chamar. Um **Tipo Abstrato de Dados** (TAD) é justamente isso: um tipo definido pelas operações que oferece, sem expor como elas são implementadas.

Operação	O que faz
`inserirInicio(v)`	coloca o valor `v` no começo
`inserirFim(v)`	coloca o valor `v` no final
`inserirNaPosicao(p, v)`	coloca `v` na posição `p`
`remover(v)`	retira a primeira ocorrência de `v`
`contem(v)`	diz se `v` está na lista
`quantidade()`	quantos elementos a lista tem
`vazia()`	diz se a lista está vazia

A mesma interface poderia ser implementada com um vetor ou com nós encadeados. Quem usa a lista não precisa saber qual das duas foi escolhida: chama as mesmas operações e recebe os mesmos resultados. O que muda é o **custo** de cada operação, e é por isso que estudamos as implementações.

3.3 O nó e a classe ListaEncadeada

O nó é uma struct com o valor e o ponteiro para o próximo, como você viu no Capítulo 1. A lista é uma classe que guarda o ponteiro para o primeiro nó, um ponteiro para o último (veremos por quê) e a quantidade de elementos:

```
// trecho: o nó e os dados da classe ListaEncadeada
struct No {
    int valor;
    No *proximo;
};

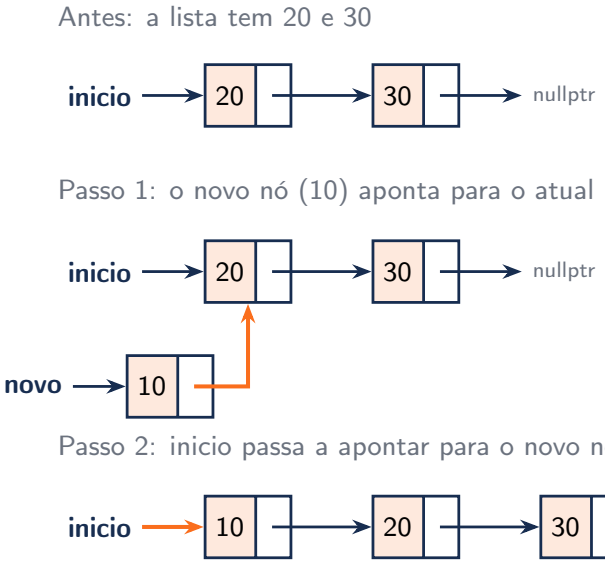
class ListaEncadeada {
private:
    No *inicio;    // primeiro nó (nullptr se a lista estiver vazia)
    No *fim;       // último nó
    int tamanho;   // quantidade de elementos
    // ... operações (a seguir)
};
```

Uma lista recém-criada não tem nós: `inicio` e `fim` valem `nullptr` e o tamanho é zero. Esse estado é a **lista vazia**, e toda operação precisa tratá-lo com cuidado.

3.4 Inserindo elementos

No início: O(1)

Inserir no início exige só dois passos, não importa quantos elementos a lista tenha: o novo nó aponta para o atual primeiro, e `inicio` passa a apontar para o novo nó.




```
// trecho da classe ListaEncadeada
void inserirInicio(int valor) {
    No *novo = new No{valor, inicio};  // passo 1: aponta para o atual
    início
    inicio = novo;                     // passo 2: início passa a ser
    o novo nó
    if (fim == nullptr) {              // a lista estava vazia:
        fim = novo;                    // o novo nó também é o último
    }
    tamanho++;
}
```

A ordem dos passos importa. Se você fizer primeiro `inicio = novo`, perde a referência para o antigo primeiro nó, e o resto da lista fica inacessível.

No fim: O(1) graças ao ponteiro fim

Sem o ponteiro `fim`, inserir no final exigiria percorrer a lista inteira até achar o último nó: $O(n)$. Guardando um ponteiro para o último, basta ligá-lo ao novo nó:

```
// trecho da classe ListaEncadeada
void inserirFim(int valor) {
    No *novo = new No{valor, nullptr};
    if (fim == nullptr) {        // lista vazia: o novo nó é o
    primeiro e o último
        inicio = novo;
```

```
        fim = novo;
    } else {
        fim->proximo = novo;     // o antigo último aponta para o novo
        fim = novo;              // e o novo passa a ser o último
    }
    tamanho++;
}
```

Esse é um exemplo de troca entre espaço e tempo: um ponteiro a mais na classe transforma uma operação $O(n)$ em $O(1)$.

Em uma posição qualquer: O(n)

Para inserir na posição `p`, é preciso caminhar até o nó da posição `p - 1` e ligar o novo nó depois dele. A caminhada custa $O(n)$; a ligação em si, $O(1)$.

```
// trecho da classe ListaEncadeada
bool inserirNaPosicao(int pos, int valor) {
    if (pos < 0 || pos > tamanho) return false;          // posição
    inválida
    if (pos == 0) { inserirInicio(valor); return true; }
    if (pos == tamanho) { inserirFim(valor); return true; }

    No *anterior = inicio;
    for (int i = 0; i < pos - 1; i++) {                  // caminha
    até pos - 1
        anterior = anterior->proximo;
    }
    anterior->proximo = new No{valor, anterior->proximo};
    tamanho++;
    return true;
}
```

3.5 Percorrendo e buscando

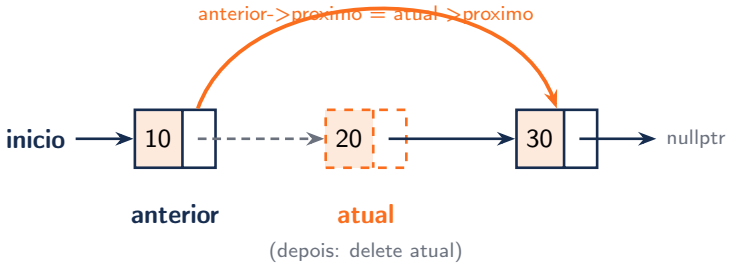
Percorrer a lista é sempre o mesmo movimento: começar em `inicio` e seguir `proximo` até chegar em `nullptr`. Buscar um valor é percorrer comparando, e por isso custa $O(n)$ no pior caso, como a busca sequencial do Capítulo 2.

```
// trecho da classe ListaEncadeada
bool contem(int valor) const {
    for (No *atual = inicio; atual != nullptr; atual = atual->proximo)
    {
        if (atual->valor == valor) {
            return true;
        }
    }
    return false;
}
```

Repare que não existe atalho para chegar ao quinto elemento: é preciso passar pelos quatro anteriores. Acessar uma posição por índice, que no vetor é $O(1)$, na lista encadeada é $O(n)$. Essa é a principal desvantagem da lista.

3.6 Removendo elementos

Para remover um nó, o nó **anterior** a ele passa a apontar para o **seguinte**. O nó removido fica fora da corrente e pode ser liberado com `delete`.



Como cada nó só conhece o próximo, a busca precisa andar com **dois ponteiros**: `atual`, que procura o valor, e `anterior`, que fica um passo atrás. A remoção ainda precisa tratar casos especiais: lista vazia, valor ausente, remoção do primeiro nó e remoção do último.

```
// trecho da classe ListaEncadeada
bool remover(int valor) {
    No *anterior = nullptr;
    No *atual = inicio;
    while (atual != nullptr && atual->valor != valor) {
        anterior = atual;
        atual = atual->proximo;
    }
    if (atual == nullptr) {              // não encontrou (ou lista
    vazia)
        return false;
    }
    if (anterior == nullptr) {           // era o primeiro nó
        inicio = atual->proximo;
    } else {                             // estava no meio ou no fim
        anterior->proximo = atual->proximo;
    }
    if (atual == fim) {                  // era o último nó
        fim = anterior;
    }
    delete atual;
    tamanho--;
    return true;
}
```

3.7 O programa completo

Juntando as operações, a classe fica assim. Repare no destrutor, que percorre a lista liberando cada nó, e nas duas linhas com `= delete`, explicadas logo depois do código.

```
struct No {
    int valor;
    No *proximo;
};

class ListaEncadeada {
private:
    No *inicio;
    No *fim;
    int tamanho;

public:
    ListaEncadeada() : inicio(nullptr), fim(nullptr), tamanho(0) {}

    // libera todos os nós quando a lista deixa de existir
    ~ListaEncadeada() {
        No *atual = inicio;
        while (atual != nullptr) {
            No *seguinte = atual->proximo;  // guarda antes de apagar
            delete atual;
            atual = seguinte;
        }
    }

    // impede cópias: duas listas não podem dividir os mesmos nós
    ListaEncadeada(const ListaEncadeada &) = delete;
    ListaEncadeada &operator=(const ListaEncadeada &) = delete;

    bool vazia() const { return tamanho == 0; }
    int quantidade() const { return tamanho; }

    void inserirInicio(int valor) {
        No *novo = new No{valor, inicio};
        inicio = novo;
        if (fim == nullptr) {
            fim = novo;
        }
        tamanho++;
    }

    void inserirFim(int valor) {
        No *novo = new No{valor, nullptr};
        if (fim == nullptr) {
            inicio = novo;
            fim = novo;
        } else {
            fim->proximo = novo;
            fim = novo;
```

```
    }
    tamanho++;
}

bool inserirNaPosicao(int pos, int valor) {
    if (pos < 0 || pos > tamanho) return false;
    if (pos == 0) { inserirInicio(valor); return true; }
    if (pos == tamanho) { inserirFim(valor); return true; }
    No *anterior = inicio;
    for (int i = 0; i < pos - 1; i++) {
        anterior = anterior->proximo;
    }
    anterior->proximo = new No{valor, anterior->proximo};
    tamanho++;
    return true;
}

bool contem(int valor) const {
    for (No *atual = inicio; atual != nullptr; atual =
    atual->proximo) {
        if (atual->valor == valor) {
            return true;
        }
    }
    return false;
}

bool remover(int valor) {
    No *anterior = nullptr;
    No *atual = inicio;
    while (atual != nullptr && atual->valor != valor) {
        anterior = atual;
        atual = atual->proximo;
    }
    if (atual == nullptr) {
        return false;
    }
    if (anterior == nullptr) {
        inicio = atual->proximo;
    } else {
        anterior->proximo = atual->proximo;
    }
    if (atual == fim) {
        fim = anterior;
    }
    delete atual;
    tamanho--;
    return true;
}

void imprimir() const {
    cout << "[ ";
```

```
        for (No *atual = inicio; atual != nullptr; atual =
        atual->proximo) {
            cout << atual->valor << " ";
        }
        cout << "]" << endl;
    }
};

int main() {
    ListaEncadeada lista;
    lista.inserirFim(20);
    lista.inserirFim(30);
    lista.inserirInicio(10);
    lista.imprimir();                       // [ 10 20 30 ]

    lista.inserirNaPosicao(2, 25);
    lista.imprimir();                       // [ 10 20 25 30 ]

    cout << boolalpha << lista.contem(25) << endl;   // true

    lista.remover(10);                      // remove o primeiro
    lista.remover(30);                      // remove o último
    lista.remover(99);                      // não existe: nada
    acontece
    lista.imprimir();                       // [ 20 25 ]
    cout << lista.quantidade() << endl;     // 2
}
```

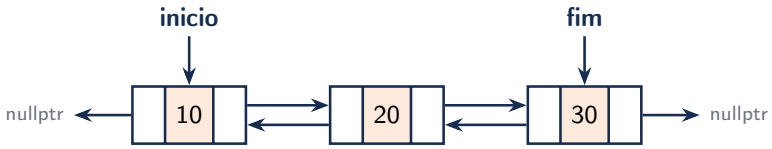
Por que proibir cópias?

Sem as linhas com `= delete`, o C++ permitiria escrever `ListaEncadeada outra = lista;`. A cópia automática copiaria só os ponteiros, e as duas listas passariam a apontar para os **mesmos nós**. Quando a primeira deixasse de existir, o destrutor liberaria os nós; quando a segunda deixasse de existir, tentaria liberá-los de novo, e o programa travaria. Proibir a cópia é a forma mais simples de evitar o problema. Uma classe que precisa ser copiável deve implementar a cópia nó por nó.

3.8 Lista duplamente encadeada

Na lista simples, cada nó só conhece o próximo. Isso tem duas consequências incômodas: não dá para percorrer a lista de trás para frente, e remover o último nó custa $O(n)$, porque é preciso encontrar o penúltimo caminhando desde o início.

A **lista duplamente encadeada** acrescenta em cada nó um ponteiro para o **anterior**:



cada nó guarda: ponteiro **anterior** | **valor** | ponteiro **próximo**

Com os dois ponteiros, inserir e remover nas duas pontas custa $O(1)$, e a lista pode ser percorrida nos dois sentidos. O preço é um ponteiro a mais por nó e mais cuidado: cada operação precisa manter `anterior` e `proximo` coerentes.

```
struct NoDuplo {
    int valor;
    NoDuplo *anterior;
    NoDuplo *proximo;
};

class ListaDupla {
private:
    NoDuplo *inicio;
    NoDuplo *fim;
    int tamanho;

public:
    ListaDupla() : inicio(nullptr), fim(nullptr), tamanho(0) {}

    ~ListaDupla() {
        while (!vazia()) {
            removerInicio();
        }
    }

    ListaDupla(const ListaDupla &) = delete;
    ListaDupla &operator=(const ListaDupla &) = delete;

    bool vazia() const { return tamanho == 0; }

    void inserirInicio(int valor) {
        NoDuplo *novo = new NoDuplo{valor, nullptr, inicio};
        if (inicio == nullptr) {
            fim = novo;
        } else {
            inicio->anterior = novo;
        }
        inicio = novo;
        tamanho++;
    }

    void inserirFim(int valor) {
```

```
        NoDuplo *novo = new NoDuplo{valor, fim, nullptr};
        if (fim == nullptr) {
            inicio = novo;
        } else {
            fim->proximo = novo;
        }
        fim = novo;
        tamanho++;
    }

    // as duas remoções são O(1): não é preciso procurar o vizinho
    bool removerInicio() {
        if (vazia()) return false;
        NoDuplo *velho = inicio;
        inicio = inicio->proximo;
        if (inicio == nullptr) {
            fim = nullptr;           // a lista ficou vazia
        } else {
            inicio->anterior = nullptr;
        }
        delete velho;
        tamanho--;
        return true;
    }

    bool removerFim() {
        if (vazia()) return false;
        NoDuplo *velho = fim;
        fim = fim->anterior;         // o penúltimo, sem precisar
        procurar
        if (fim == nullptr) {
            inicio = nullptr;
        } else {
            fim->proximo = nullptr;
        }
        delete velho;
        tamanho--;
        return true;
    }

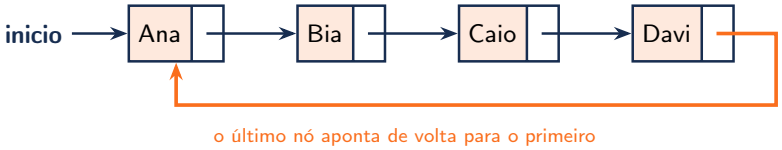
    void imprimir() const {
        for (NoDuplo *a = inicio; a != nullptr; a = a->proximo) cout
        << a->valor << " ";
        cout << endl;
    }

    void imprimirInvertido() const {
        for (NoDuplo *a = fim; a != nullptr; a = a->anterior) cout <<
        a->valor << " ";
        cout << endl;
    }
};
```

```
int main() {
    ListaDupla lista;
    lista.inserirFim(20);
    lista.inserirFim(30);
    lista.inserirInicio(10);
    lista.imprimir();             // 10 20 30
    lista.imprimirInvertido();    // 30 20 10
    lista.removerFim();
    lista.removerInicio();
    lista.imprimir();             // 20
}
```

3.9 Lista circular

Em uma **lista circular**, o último nó não aponta para `nullptr`: aponta de volta para o primeiro. A lista vira um anel, sem começo nem fim naturais.



Ela é útil sempre que os elementos são atendidos em rodízio, voltando ao primeiro depois do último: turnos de um jogo, a fila de processos que dividem o processador, as músicas de uma playlist em repetição.

Um detalhe elegante: basta guardar um ponteiro para o **último** nó. O primeiro é sempre `ultimo->proximo`, então as duas pontas ficam acessíveis em $O(1)$ com um único ponteiro.

```
struct Jogador {
    string nome;
    Jogador *proximo;
};

class Rodizio {
private:
    Jogador *ultimo = nullptr;   // ultimo->proximo é o primeiro

public:
    ~Rodizio() {
        if (ultimo == nullptr) return;
        Jogador *atual = ultimo->proximo;
        ultimo->proximo = nullptr;          // abre o anel para poder
        parar
        while (atual != nullptr) {
            Jogador *seguinte = atual->proximo;
```

```
            delete atual;
            atual = seguinte;
        }
    }

    Rodizio() = default;
    Rodizio(const Rodizio &) = delete;
    Rodizio &operator=(const Rodizio &) = delete;

    void adicionar(const string &nome) {
        Jogador *novo = new Jogador{nome, nullptr};
        if (ultimo == nullptr) {
            novo->proximo = novo;           // um só jogador: aponta
            para si mesmo
        } else {
            novo->proximo = ultimo->proximo; // o novo aponta para o
            primeiro
            ultimo->proximo = novo;          // o antigo último aponta
            para o novo
        }
        ultimo = novo;                       // o novo passa a ser o
        último
    }

    void jogar(int turnos) const {
        if (ultimo == nullptr) return;
        Jogador *vez = ultimo->proximo;      // começa pelo primeiro
        for (int t = 1; t <= turnos; t++) {
            cout << "Turno " << t << ": " << vez->nome << endl;
            vez = vez->proximo;              // depois do último,
            volta ao primeiro
        }
    }
};

int main() {
    Rodizio jogo;
    jogo.adicionar("Ana");
    jogo.adicionar("Bia");
    jogo.adicionar("Caio");
    jogo.jogar(5);   // Ana, Bia, Caio, Ana, Bia
}
```

Cuidado com a parada: em uma lista circular, percorrer até encontrar `nullptr` gera um laço infinito. Por isso o percurso para por contagem (como em `jogar`) ou ao voltar ao nó de partida, e o destrutor "abre" o anel antes de liberar os nós.

3.10 Vetor ou lista encadeada?

Nenhuma das duas estruturas é melhor em tudo. A escolha depende de quais operações o seu problema faz com mais frequência:

Operação	Vetor	Lista simples	Lista dupla
Acessar a posição i	$O(1)$	$O(n)$	$O(n)$
Inserir no início	$O(n)$	$O(1)$	$O(1)$
Inserir no fim	$O(1)$*	$O(1)$	$O(1)$
Remover do início	$O(n)$	$O(1)$	$O(1)$
Remover do fim	$O(1)$	$O(n)$	$O(1)$
Buscar um valor	$O(n)$	$O(n)$	$O(n)$
Memória extra por elemento	nenhuma	1 ponteiro	2 ponteiros

* No vetor dinâmico, quase sempre $O(1)$; de vez em quando, quando enche, a cópia para um espaço maior custa $O(n)$. Na média, continua $O(1)$: dizemos que é $O(1)$ **amortizado**.

Em resumo: prefira o **vetor** quando o acesso por posição for frequente e as inserções acontecerem no fim; prefira a **lista** quando houver muitas inserções e remoções nas pontas ou no meio de uma sequência que você já está percorrendo. Na prática, vetores também se beneficiam de estarem contínuos na memória, o que o processador aproveita muito bem; por isso, para listas pequenas, o vetor costuma ser mais rápido mesmo quando a tabela sugere o contrário.

Comparando com Python

A `list` do Python é um vetor dinâmico, não uma lista encadeada: acessar `lista[i]` é $O(1)$, mas `lista.insert(0, x)` é $O(n)$. Quando você precisa inserir e remover nas duas pontas, a escolha certa é o `deque`, do módulo `collections`, implementado com blocos encadeados. E, se quiser, dá para escrever uma lista encadeada em Python com a mesma ideia de nós:

```
from collections import deque

class No:
    def __init__(self, valor, proximo=None):
        self.valor = valor
        self.proximo = proximo     # referência para o próximo nó (ou
        None)

inicio = No(10, No(20, No(30)))   # 10 -> 20 -> 30
atual = inicio
while atual is not None:
    print(atual.valor, end=" ")
    atual = atual.proximo
print()

pedidos = deque([20, 30])
pedidos.appendleft(10)           # O(1), como inserirInicio
pedidos.append(40)               # O(1), como inserirFim
print(pedidos.popleft())         # 10, O(1)
```

Em Python não há `delete`: quando nenhum nó aponta mais para um objeto, a coleta de lixo o remove.

A biblioteca padrão do C++ também oferece listas prontas: `std::forward_list` (simplesmente encadeada) e `std::list` (duplamente encadeada). Agora que você sabe como funcionam por dentro, use-as nos seus projetos.

Erros Comuns

- **Perder o resto da lista.** Ao inserir no início, atualizar `inicio` antes de ligar o novo nó ao antigo primeiro deixa todos os outros nós inacessíveis.
- **Esquecer a lista vazia.** Toda operação precisa funcionar quando `inicio` é `nullptr`. Acessar `inicio->valor` em uma lista vazia derruba o programa.
- **Não atualizar o ponteiro `fim`.** Ao remover o último nó, `fim` precisa passar a apontar para o anterior (ou para `nullptr`, se a lista ficou vazia).
- **Usar o nó depois do `delete`.** No destrutor, guarde o `proximo` em uma variável *antes* de liberar o nó atual.
- **Esquecer o destrutor.** Sem ele, todos os nós ficam presos na memória quando a lista deixa de existir.
- **Copiar uma lista sem tratar a cópia.** A cópia automática divide os mesmos nós entre duas listas e causa `delete` duplo. Proíba a cópia ou implemente-a nó por nó.
- **Esquecer um dos ponteiros na lista dupla.** Cada inserção e remoção precisa atualizar `anterior` e `proximo` dos vizinhos.
- **Laço infinito na lista circular.** Na lista circular, nenhum nó aponta para `nullptr`: o percurso precisa parar por contagem ou ao voltar ao início.

Capítulo 4 – Pilhas e Filas

A lista encadeada do Capítulo 3 permite inserir e remover em qualquer ponto. Muitos problemas, porém, precisam do contrário: de uma estrutura que **restringe** onde se pode inserir e remover. Essas restrições não são uma limitação, e sim uma garantia de ordem. Neste capítulo você vai construir as duas estruturas restritas mais usadas da computação: a **pilha** e a **fila**. Ambas serão genéricas, com templates (Capítulo 1), e construídas com nós encadeados (Capítulo 3).

4.1 Estruturas com regras de acesso

Pense em uma pilha de pratos: você coloca um prato sempre em cima e, quando precisa de um, pega o de cima. O último prato colocado é o primeiro a ser usado. Agora pense na fila de um banco: quem chega vai para o final, e quem é atendido sai pela frente. O primeiro a chegar é o primeiro a sair.

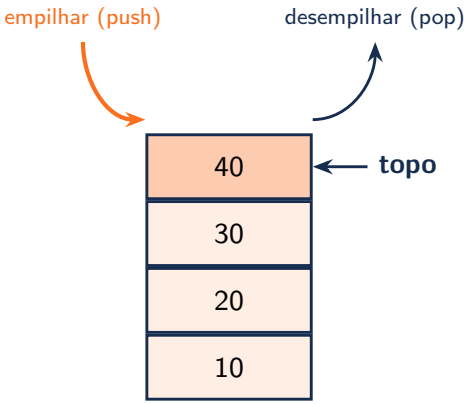
Essas duas regras têm nomes:

- **LIFO** (*Last In, First Out*, o último a entrar é o primeiro a sair): é a regra da **pilha**.
- **FIFO** (*First In, First Out*, o primeiro a entrar é o primeiro a sair): é a regra da **fila**.

Você usa as duas o tempo todo, mesmo sem perceber. O botão de desfazer (Ctrl+Z) de um editor é uma pilha: desfaz primeiro a última ação. Os documentos enviados para uma impressora formam uma fila: são impressos na ordem em que chegaram.

4.2 A pilha

Em uma pilha, todas as operações acontecem em uma única ponta, chamada **topo**.



o último a entrar é o primeiro a sair (LIFO)

Operação	O que faz	Custo
`empilhar(v)`	coloca `v` no topo (*push*)	$O(1)$
`desempilhar()`	retira e devolve o topo (*pop*)	$O(1)$
`espiar()`	devolve o topo sem retirar (*top*)	$O(1)$
`vazia()`	diz se não há elementos	$O(1)$
`quantidade()`	quantos elementos há	$O(1)$

4.3 Pilha com nós encadeados

Na lista encadeada, inserir e remover no **início** custa $O(1)$. Então basta usar o início da lista como topo da pilha: empilhar é inserir no início, desempilhar é remover do início. Cada nó aponta para o que está **abaixo** dele.

Repare em duas novidades. O nó agora fica **dentro** da classe, porque só a pilha precisa conhecê-lo. E desempilhar uma pilha vazia é um erro de quem chama: em vez de devolver um valor inventado, a função **lança uma exceção** com `throw`, que interrompe a operação e pode ser tratada com `try`/`catch`.

```
#include <stdexcept>   // runtime_error

template <typename T>
class Pilha {
private:
    struct No {
        T valor;
        No *abaixo;    // o nó que está embaixo deste
    };
    No *topo = nullptr;
    int tamanho = 0;

public:
    Pilha() = default;
    ~Pilha() {
        while (!vazia()) {
            desempilhar();
        }
    }
    Pilha(const Pilha &) = delete;
    Pilha &operator=(const Pilha &) = delete;

    bool vazia() const { return topo == nullptr; }
    int quantidade() const { return tamanho; }

    void empilhar(const T &valor) {
        topo = new No{valor, topo};   // o novo nó fica por cima do
        antigo topo
        tamanho++;
    }

    T desempilhar() {
        if (vazia()) {
```

```
            throw runtime_error("pilha vazia");
        }
        No *velho = topo;
        T valor = velho->valor;
        topo = topo->abaixo;
        delete velho;
        tamanho--;
        return valor;
    }

    const T &espiar() const {
        if (vazia()) {
            throw runtime_error("pilha vazia");
        }
        return topo->valor;
    }
};

int main() {
    Pilha<int> p;
    for (int v = 10; v <= 40; v += 10) {
        p.empilhar(v);                        // 10, 20, 30, 40
    }
    cout << "Topo: " << p.espiar() << endl;  // 40
    cout << p.desempilhar() << endl;         // 40
    cout << p.desempilhar() << endl;         // 30
    cout << "Restam " << p.quantidade() << endl;  // 2

    Pilha<string> vazia;
    try {
        vazia.desempilhar();
    } catch (const runtime_error &erro) {
        cout << "Erro: " << erro.what() << endl;  // Erro: pilha vazia
    }
}
```

Como a classe é um template, a mesma pilha serve para `int`, `string`, `char` ou qualquer outro tipo. Todas as operações são $O(1)$: nenhuma percorre a estrutura.

4.4 Aplicações da pilha

Verificar se os parênteses estão balanceados

Compiladores e editores de código precisam conferir se cada `(`, `[` e `{` foi fechado na ordem certa. A expressão `{[(2+3)*4]}` está correta; `(2+3]` e `((1+2)` não estão.

A pilha resolve isso de forma natural: cada símbolo que abre é empilhado; cada símbolo que fecha precisa combinar com o topo. O último aberto deve ser o primeiro fechado, exatamente a regra LIFO.

```
// trecho: usa a classe Pilha da seção 4.3
bool balanceado(const string &expressao) {
    Pilha<char> abertos;
    for (char c : expressao) {
        if (c == '(' || c == '[' || c == '{') {
            abertos.empilhar(c);
        } else if (c == ')' || c == ']' || c == '}') {
            if (abertos.vazia()) {
                return false;                // fechou algo que não
                foi aberto
            }
            char a = abertos.desempilhar();
            if ((c == ')' && a != '(') || (c == ']' && a != '[') || (c
            == '}' && a != '{')) {
                return false;                // fechou com o tipo
                errado
            }
        }
    }
    return abertos.vazia();                  // sobrou algo aberto?
}
```

Calcular expressões em notação pós-fixa

Na notação que usamos no dia a dia, o operador fica entre os números: $(3 + 4) \cdot 2$. Na **notação pós-fixa**, o operador vem depois dos operandos: `3 4 + 2 *`. Ela dispensa parênteses e é muito fácil de calcular com uma pilha, por isso calculadoras e compiladores a usam internamente.

A regra é: número, empilha; operador, desempilha dois números, faz a conta e empilha o resultado. No final, o resultado é o único valor na pilha.

Lido	Ação	Pilha (topo à direita)
`3`	empilha	3
`4`	empilha	3 4
`+`	desempilha 4 e 3, empilha $3 + 4$	7
`2`	empilha	7 2
`*`	desempilha 2 e 7, empilha $7 \cdot 2$	14

```
// trecho: usa a classe Pilha da seção 4.3
#include <sstream>   // istringstream: lê a expressão palavra por
palavra

double avaliarPosfixa(const string &expressao) {
    Pilha<double> p;
    istringstream entrada(expressao);
    string item;
    while (entrada >> item) {
        if (item == "+" || item == "-" || item == "*" || item == "/")
        {
```

```
            double b = p.desempilhar();   // o segundo operando sai
            primeiro
            double a = p.desempilhar();
            if (item == "+") p.empilhar(a + b);
            else if (item == "-") p.empilhar(a - b);
            else if (item == "*") p.empilhar(a * b);
            else p.empilhar(a / b);
        } else {
            p.empilhar(stod(item));       // converte o texto em
            número
        }
    }
    return p.desempilhar();
}
```

Veja as duas aplicações funcionando juntas. Para o programa ficar mais curto, a pilha aparece em versão resumida, só com as operações usadas aqui:

```
#include <stdexcept>
#include <sstream>

template <typename T>
class Pilha {
private:
    struct No {
        T valor;
        No *abaixo;
    };
    No *topo = nullptr;

public:
    Pilha() = default;
    ~Pilha() { while (!vazia()) desempilhar(); }
    Pilha(const Pilha &) = delete;
    Pilha &operator=(const Pilha &) = delete;

    bool vazia() const { return topo == nullptr; }
    void empilhar(const T &valor) { topo = new No{valor, topo}; }
    T desempilhar() {
        if (vazia()) throw runtime_error("pilha vazia");
        No *velho = topo;
        T valor = velho->valor;
        topo = topo->abaixo;
        delete velho;
        return valor;
    }
};

bool balanceado(const string &expressao) {
    Pilha<char> abertos;
    for (char c : expressao) {
```

```
        if (c == '(' || c == '[' || c == '{') {
            abertos.empilhar(c);
        } else if (c == ')' || c == ']' || c == '}') {
            if (abertos.vazia()) {
                return false;
            }
            char a = abertos.desempilhar();
            if ((c == ')' && a != '(') || (c == ']' && a != '[') || (c
            == '}' && a != '{')) {
                return false;
            }
        }
    }
    return abertos.vazia();
}

double avaliarPosfixa(const string &expressao) {
    Pilha<double> p;
    istringstream entrada(expressao);
    string item;
    while (entrada >> item) {
        if (item == "+" || item == "-" || item == "*" || item == "/")
        {
            double b = p.desempilhar();
            double a = p.desempilhar();
            if (item == "+") p.empilhar(a + b);
            else if (item == "-") p.empilhar(a - b);
            else if (item == "*") p.empilhar(a * b);
            else p.empilhar(a / b);
        } else {
            p.empilhar(stod(item));
        }
    }
    return p.desempilhar();
}

int main() {
    cout << boolalpha;
    cout << balanceado("{[(2+3)*4]}") << endl;   // true
    cout << balanceado("(2+3]") << endl;         // false
    cout << balanceado("((1+2)") << endl;        // false
    cout << balanceado(")(") << endl;            // false

    cout << avaliarPosfixa("3 4 + 2 *") << endl;          // 14
    cout << avaliarPosfixa("5 1 2 + 4 * + 3 -") << endl;  // 14
}
```

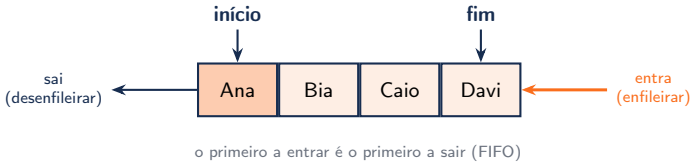
A pilha de execução

Você já conhece uma pilha muito importante: a **pilha de execução** do programa, citada nos Capítulos 1 e 2. Cada vez que uma função é chamada, um espaço com suas variáveis é empilhado; quando a função termina, esse espaço é desempilhado e o

programa volta ao ponto em que estava. Por isso uma recursão sem fim causa o erro conhecido como *stack overflow* (estouro da pilha): as chamadas são empilhadas até não caber mais nenhuma.

4.5 A fila

Em uma fila, os elementos entram por uma ponta (o **fim**) e saem pela outra (o **início**).



Operação	O que faz	Custo
`enfileirar(v)`	coloca `v` no fim (*enqueue*)	$O(1)$
`desenfileirar()`	retira e devolve o início (*dequeue*)	$O(1)$
`frente()`	devolve o início sem retirar	$O(1)$
`vazia()`	diz se não há elementos	$O(1)$

4.6 Fila com nós encadeados

A lista encadeada com ponteiro para o fim (Capítulo 3) já tem tudo que a fila precisa: inserir no fim e remover do início, ambos em $O(1)$.

```
#include <stdexcept>

template <typename T>
class Fila {
private:
    struct No {
        T valor;
        No *proximo;
    };
    No *inicio = nullptr;   // de onde sai
    No *fim = nullptr;      // por onde entra
    int tamanho = 0;

public:
    Fila() = default;
    ~Fila() {
        while (!vazia()) {
            desenfileirar();
        }
    }
    Fila(const Fila &) = delete;
    Fila &operator=(const Fila &) = delete;
```

```
    bool vazia() const { return inicio == nullptr; }
    int quantidade() const { return tamanho; }

    void enfileirar(const T &valor) {
        No *novo = new No{valor, nullptr};
        if (fim == nullptr) {
            inicio = novo;          // fila vazia: o novo é o primeiro
        } else {
            fim->proximo = novo;    // entra atrás do último
        }
        fim = novo;
        tamanho++;
    }

    T desenfileirar() {
        if (vazia()) {
            throw runtime_error("fila vazia");
        }
        No *velho = inicio;
        T valor = velho->valor;
        inicio = inicio->proximo;
        if (inicio == nullptr) {
            fim = nullptr;          // a fila ficou vazia
        }
        delete velho;
        tamanho--;
        return valor;
    }

    const T &frente() const {
        if (vazia()) {
            throw runtime_error("fila vazia");
        }
        return inicio->valor;
    }
};

int main() {
    Fila<string> atendimento;
    atendimento.enfileirar("Ana");
    atendimento.enfileirar("Bia");
    atendimento.enfileirar("Caio");

    cout << "Proximo a ser atendido: " << atendimento.frente() <<
    endl;  // Ana
    while (!atendimento.vazia()) {
        cout << "Atendendo " << atendimento.desenfileirar() << endl;
        // Ana, Bia, Caio
    }
}
```

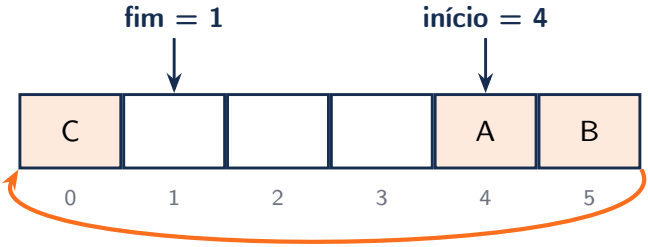
Repare no cuidado em `desenfileirar`: quando o último elemento sai, `fim` também precisa voltar a `nullptr`. Se isso for esquecido, o próximo `enfileirar` vai ligar o novo

nó a um nó que já foi apagado.

4.7 Fila circular em vetor

Também é possível fazer uma fila com um vetor de tamanho fixo, o que evita alocar um nó a cada inserção. A ideia ingênua seria remover sempre da posição 0 e deslocar os demais elementos, mas isso custaria $O(n)$ a cada saída.

A solução é a **fila circular**: em vez de mover os elementos, movemos os índices. `inicio` indica onde está o primeiro elemento e `fim` indica onde entra o próximo. Quando um índice passa da última posição, ele volta para a posição 0, como se o vetor fosse um anel. Quem faz essa volta é o operador de resto da divisão, `%`.



Na figura, a fila tem capacidade 6 e contém, na ordem de saída, A (posição 4), B (posição 5) e C (posição 0). O próximo elemento entrará na posição 1.

```
#include <stdexcept>

template <typename T>
class FilaCircular {
private:
    T *dados;
    int capacidade;
    int inicio = 0;    // posição do primeiro elemento
    int fim = 0;       // posição onde entra o próximo
    int tamanho = 0;

public:
    explicit FilaCircular(int cap) : dados(new T[cap]),
    capacidade(cap) {}
    ~FilaCircular() { delete[] dados; }
    FilaCircular(const FilaCircular &) = delete;
    FilaCircular &operator=(const FilaCircular &) = delete;

    bool vazia() const { return tamanho == 0; }
    bool cheia() const { return tamanho == capacidade; }

    bool enfileirar(const T &valor) {
```

```
        if (cheia()) {
            return false;                    // não há espaço
        }
        dados[fim] = valor;
        fim = (fim + 1) % capacidade;        // depois da última,
        volta para 0
        tamanho++;
        return true;
    }

    T desenfileirar() {
        if (vazia()) {
            throw runtime_error("fila vazia");
        }
        T valor = dados[inicio];
        inicio = (inicio + 1) % capacidade;
        tamanho--;
        return valor;
    }
};

int main() {
    FilaCircular<int> f(6);
    for (int v = 1; v <= 6; v++) {
        f.enfileirar(v);                     // ocupa as posições
        0 a 5
    }
    cout << boolalpha << f.enfileirar(7) << endl;  // false: está
    cheia

    cout << f.desenfileirar() << " ";        // 1
    cout << f.desenfileirar() << endl;       // 2

    f.enfileirar(7);                         // entra na posição 0
    (deu a volta)
    f.enfileirar(8);                         // entra na posição 1
    while (!f.vazia()) {
        cout << f.desenfileirar() << " ";    // 3 4 5 6 7 8
    }
    cout << endl;
}
```

A fila circular faz todas as operações em $O(1)$ e não aloca memória a cada inserção, mas tem capacidade fixa. É muito usada em sistemas que recebem dados continuamente, como teclados, placas de rede e players de áudio, onde é chamada de *buffer* circular.

4.8 Aplicações da fila

- **Atendimento por ordem de chegada**: senhas de banco, pedidos de uma lanchonete, chamados de suporte.
- **Fila de impressão**: os documentos são impressos na ordem em que foram enviados.

- **Escalonamento de processos**: o sistema operacional reveza o processador entre os programas usando filas.
- **Busca em largura em grafos**: no Capítulo 6, a fila vai garantir que um grafo seja explorado em camadas, primeiro os vizinhos mais próximos.

4.9 Pilhas e filas prontas

A biblioteca padrão do C++ oferece `stack` (pilha), `queue` (fila) e `deque`, uma fila de duas pontas que permite inserir e remover no início e no fim. Atenção a um detalhe: no C++, `pop()` retira o elemento, mas **não o devolve**. Para ler o valor, use `top()` (pilha) ou `front()` (fila) antes de chamar `pop()`.

```
#include <stack>
#include <queue>

int main() {
    stack<int> pilha;
    pilha.push(10);
    pilha.push(20);
    cout << pilha.top() << endl;    // 20: lê o topo
    pilha.pop();                    // retira, sem devolver

    queue<string> fila;
    fila.push("Ana");
    fila.push("Bia");
    cout << fila.front() << endl;   // Ana: lê o início
    fila.pop();
    cout << fila.size() << endl;    // 1
}
```

Comparando com Python

Em Python, a própria `list` funciona bem como pilha, porque `append` e `pop` no fim custam $O(1)$. Para fila, porém, **não** use a lista: `lista.pop(0)` desloca todos os elementos e custa $O(n)$. Use o `deque`:

```
from collections import deque

pilha = []
pilha.append(10)
pilha.append(20)
print(pilha.pop())        # 20, O(1)

fila = deque()
fila.append("Ana")        # enfileirar, O(1)
fila.append("Bia")
print(fila.popleft())     # Ana, desenfileirar em O(1)
```

Erros Comuns

- **Desempilhar ou desenfileirar uma estrutura vazia.** Sempre verifique `vazia()` antes, ou trate a exceção. Na STL do C++, chamar `top()` ou `pop()` em uma pilha vazia tem comportamento indefinido.
- **Esperar que o `pop()` da STL devolva o valor.** Ele só retira. Leia com `top()` ou `front()` antes.
- **Inverter a ordem dos operandos.** Na pós-fixa, o primeiro valor desempilhado é o *segundo* operando: em `8 2 -`, a conta é $8 - 2$, não $2 - 8$.
- **Não zerar o `fim` quando a fila esvazia.** Ao retirar o último elemento, `fim` deve voltar a `nullptr`.
- **Esquecer o resto da divisão na fila circular.** Sem `% capacidade`, o índice passa do fim do vetor e o programa escreve fora da memória reservada.
- **Confundir fila cheia com fila vazia.** Na fila circular, `inicio == fim` acontece nos dois casos. Por isso guardamos o `tamanho`.
- **Usar `lista.pop(0)` como fila em Python.** Cada saída custa $O(n)$; com muitos elementos, o programa fica lento. Use `deque`.